subscribe to a channel by name, the server then sends messages to that channel, and Rails knows to direct those messages across the correct WebSockets. When using Turbo, the turbo-rails gem does a lot of the behind-the-scenes work with a Turbo::StreamsChannel class and a lot of helper methods that access that class.

On our client side, we need to register that our view wants to receive messages in a channel—our schedule page wants to receive messages that a concert has been moved to or from the current user's favorite status. There's a helper method for that:

chapter_09/01/app/views/schedules/show.html.erb

```
<%= turbo_stream_from(current_user, :favorites) %>
```

The helper is turbo_stream_from, and it takes as an argument an arbitrary list of items that are put together to create the name of the stream being listened to. Each of these calls connects you to one stream—multiple arguments mean the name of the stream has compound parts. To listen to multiple streams, you need to call turbo_stream_from multiple times.

In our case, we have two parts to the stream name: the current_user and the symbol :favorites. The turbo-rails gem basically concatenates all the bits together, but if the object defines something meaningful with the methods to_param or to_gid_param, it uses that value. The most common use case is to use an ActiveRecord object in the stream name and use the object's ID to differentiate it from other objects. For example, for current_user, that value will be something like user:5, giving us a full name of user:5:favorite, based on the user ID. This means that if you log in as another user and open a browser as that user, the two ActionCables will be distinct as far as the server is concerned.

There's one more part to the stream name, which is that turbo-rails digitally signs the names so that the user ID number isn't actually leaked to the browser in plain text, making the names harder to spoof. A side effect of this name signing is that on the server side, we need to use turbo-rails helpers to interact with the ActionCable because they will automatically do the same name signing; otherwise, the names of the streams won't match and the message won't be